

AI-Powered Smart Water Reservoir Management and Demand Forecasting Platform

COURSE NAME: SOFTWARE ENGINEERING

COURSE CODE: CSA1013

Submitted by:
VISHAL B
192524485
B.TECH AIDS

1. Industry Context and Problem Statement

1.1 Industry Context

Water reservoirs are essential infrastructure for supplying drinking water, irrigation, and industrial use. Increasing climate variability — manifesting as erratic monsoons, prolonged droughts, and sudden flood events — combined with steadily rising population and industrial demand, creates persistent challenges in balancing water availability against consumption. Artificial intelligence, the Internet of Things (IoT), satellite-based remote sensing, and cloud analytics have matured to the point where they can jointly support intelligent, proactive water-resource management rather than reactive, manual oversight.

Reservoir authorities today typically rely on periodic manual readings, spreadsheet-based recordkeeping, and experience-driven judgment calls. This approach performs adequately under stable conditions but degrades sharply during extreme events, when timely, data-backed decisions matter most. A digital platform that continuously ingests sensor, weather, and consumption data — and converts it into forecasts and recommendations — directly addresses this gap.

1.2 Problem Statement

Traditional reservoir management relies on historical observations and manual decision-making, often leading to inefficient water allocation during droughts or heavy rainfall. An AI-powered reservoir management platform should monitor reservoir levels, rainfall forecasts, inflow and outflow rates, and water demand patterns. The system should predict future water availability, recommend optimal distribution strategies, and support authorities in sustainable water resource planning.

1.3 Scope of This Report

This report addresses the architecture design assignment for Problem Statement 34. It analyzes system requirements, selects and justifies a software architecture, presents an architecture diagram, describes components and their interactions,

evaluates quality attributes, discusses security and deployment concerns, and closes with a justification of architectural decisions, a risk assessment, and a roadmap for future enhancement.

2. Objectives and Expected Outcomes

2.1 Project Objectives

The platform's objectives, derived directly from the problem statement, are to:

- Monitor reservoir water levels continuously using IoT sensors and SCADA integration.
- Forecast short- and long-term water demand across drinking, irrigation, and industrial users.
- Predict future water availability from inflow, outflow, and rainfall data.
- Optimize water distribution and release strategies under normal and drought conditions.
- Analyze rainfall patterns and satellite data for climate-aware planning.
- Generate water management reports for reservoir authorities and government agencies.
- Support drought planning through early-warning and scenario-based decision support.
- Improve overall water resource utilization and reduce wastage across the supply chain.

2.2 Expected Outcomes

A successful deployment of the platform is expected to deliver the following outcomes:

- Efficient water allocation across drinking, irrigation, and industrial demand segments.
- Improved day-to-day and long-term reservoir management practices.
- Reduced water shortages through earlier and more accurate demand-supply forecasting.
- Better drought preparedness via scenario planning and early-warning alerts.
- Sustainable water utilization aligned with long-term climate and population trends.

- More accurate demand forecasting than historical, spreadsheet-based estimation.
- Enhanced environmental conservation through reduced over-extraction and waste.
- Improved public water supply reliability and transparency for citizens and authorities.

3. Analysis of System Requirements

3.1 Stakeholder Analysis

The platform serves a diverse stakeholder base, each with distinct needs that shape the architecture:

Stakeholder	Primary Needs
Reservoir Authorities / Operators	Real-time monitoring, release-schedule recommendations, drought/flood alerts, audit trails
Government / Irrigation Boards	Aggregated reports, policy-support analytics, cross-reservoir visibility, compliance data
Field Officers	Mobile access to live readings, sensor health status, and task/alert notifications
Farmers / Irrigation Users	Advance notice of water release schedules and irrigation allocations
General Public	Transparent public portal showing reservoir status and supply outlook
System Administrators	User/role management, integration monitoring, platform configuration
Disaster Management Agencies	Flood/drought early-warning feeds and API-based data exchange

3.2 Functional Requirements

- Real-time acquisition of reservoir level, inflow/outflow, and rainfall data from sensors, SCADA, and satellite feeds.
- Time-series forecasting of water demand and availability using historical and live data.

- Optimization engine that recommends distribution and release schedules under normal and drought scenarios.
- Rule-based and ML-based alerting for critical thresholds (flood risk, drought risk, sensor failure).
- Dashboards and scheduled reports for reservoir authorities, and a public portal for water-supply transparency.
- Role-based access control for administrators, field officers, and government agencies, with a full audit trail of decisions.
- Integration with external meteorological, irrigation-board, and disaster-management systems via open APIs.
- Scenario simulation allowing authorities to model 'what-if' release and allocation strategies before committing.
- Historical data archival and retrieval to support long-term trend analysis and model retraining.
- Mobile and web access with offline-tolerant data capture for field officers in low-connectivity areas.

3.3 Non-Functional Requirements

Quality Attribute	Requirement Driven by the Problem Statement
Scalability	Must ingest data from a growing number of reservoirs, sensors, and satellite tiles without redesign; forecasting workloads must scale independently of ingestion.
Maintainability	Forecasting models and rules evolve with climate patterns; modules must be updatable without full-system redeployment.
Reliability	Drought/flood alerts must be delivered correctly even under partial network or sensor failure; no single point of failure in critical paths.
Availability	Monitoring and alerting must remain operational 24/7, since delayed drought/flood response has real-world consequences.

Quality Attribute	Requirement Driven by the Problem Statement
Performance	Sensor and rainfall data must be processed near real-time; forecasts should refresh on defined cycles (e.g., hourly/daily) without lag.
Security	Reservoir control interfaces (SCADA) and government data exchanges require strong authentication, encryption, and access control.
Usability	Dashboards must present complex forecasts and recommendations in an accessible form for non-technical authority staff.
Interoperability	Must integrate cleanly with heterogeneous third-party meteorological, SCADA, and government systems using standard protocols.

3.4 Constraints and Assumptions

- Reservoirs may be in remote locations with intermittent connectivity, requiring edge buffering.
- Sensor and SCADA vendors vary across sites, requiring protocol-agnostic ingestion.
- Government data-sharing requirements mandate auditable, access-controlled exchange.
- Historical data quality varies by reservoir, which the platform must accommodate during model training.

4. Selected Software Architecture

4.1 Architectural Style

A Hybrid Architecture is selected, combining an Edge/IoT layer for field data capture, an Event-Driven backbone (message queue / event bus) for real-time data streaming, and a Microservices architecture for the core business logic (monitoring, forecasting, optimization, alerting, reporting), all exposed through an API Gateway to web, mobile, and public-data clients.

4.2 Alternative Architectures Considered

Alternative	Why It Was Not Selected
Monolithic architecture	Tightly couples monitoring, forecasting, and reporting; a fault or slow ML job in one area risks degrading unrelated functions such as alerting.
Layered (n-tier) architecture	Simpler to build, but forces monitoring, forecasting, and reporting into one deployable unit, making independent scaling of ML workloads and real-time alerting difficult.
Pure request/response microservices (no event bus)	Would require services to poll or synchronously call each other for telemetry, adding latency and tight coupling unsuitable for continuous sensor streams.
Serverless-only architecture	Attractive for bursty workloads, but cold-start latency and execution-time limits are a poor fit for always-on monitoring and stateful stream processing.

4.3 Justification for Selection

- **Independently scalable workloads:** Sensor ingestion, ML forecasting, and reporting have very different load profiles (continuous high-frequency telemetry vs. periodic batch forecasting). Microservices let each be scaled independently, which a single monolith or pure layered design cannot do efficiently.
- **Real-time responsiveness:** Reservoir levels, rainfall, and inflow/outflow change continuously. An event-driven backbone (e.g., Kafka/MQTT)

decouples producers (sensors, satellites) from consumers (forecasting, alerting), enabling near real-time processing without tight coupling.

- **Heterogeneous data sources:** IoT sensors, SCADA systems, satellite feeds, and government APIs use different protocols and cadences. An edge/gateway layer normalizes this heterogeneity before it enters the core system.
- **Fault isolation:** If the forecasting service fails, monitoring and alerting continue to operate — critical for a system where drought/flood alerts are safety-relevant. A monolithic or purely layered design would risk cascading failure.
- **Extensibility for future growth:** New services (e.g., a canal-automation module or a groundwater-integration service) can be added as independent microservices consuming existing event streams, without touching existing code — directly supporting the objective of long-term sustainable water planning.

Trade-off considered: a simpler Layered (n-tier) architecture was evaluated first, as it is easier to build and deploy for a single-team project. It was rejected because it tightly couples monitoring, forecasting, and reporting in one deployable unit, making independent scaling of ML workloads and real-time alerting difficult — both essential to this problem domain.

5. Software Architecture Diagram

The diagram below shows six layers — field/edge data sources, data ingestion and event streaming, core microservices, AI/ML analytics, data storage, and application/presentation — together with the external systems the platform integrates with.

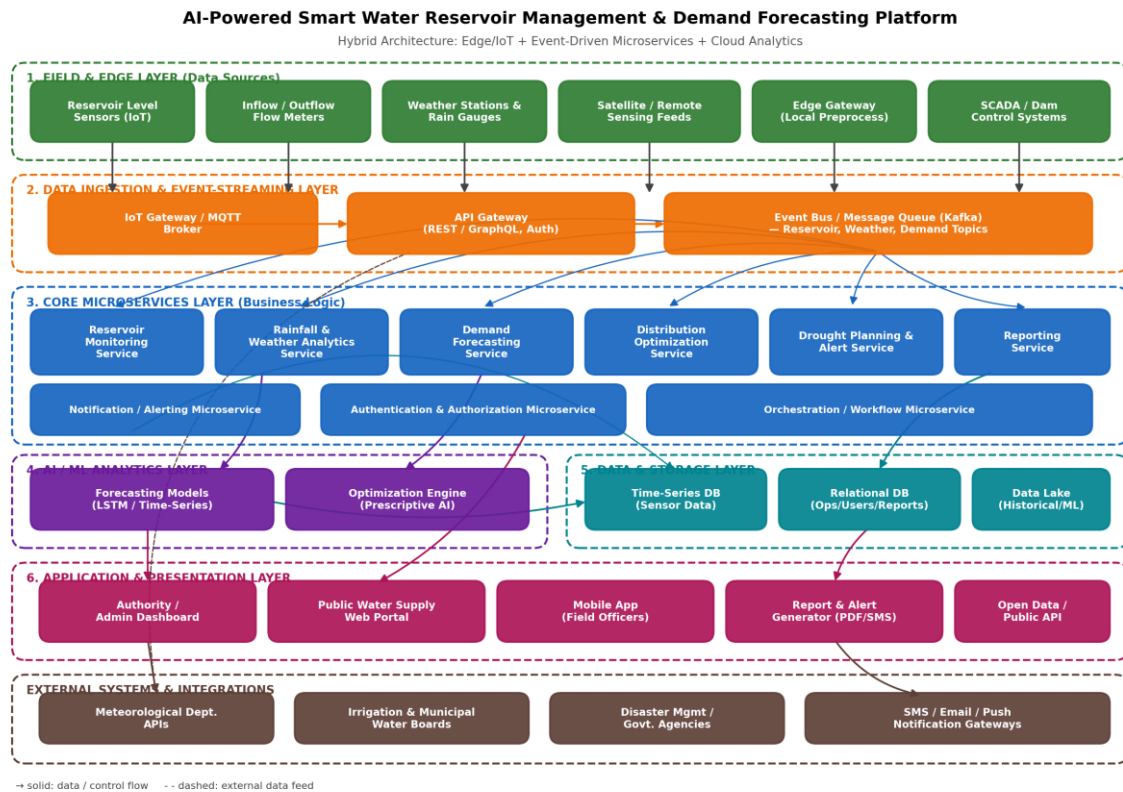


Figure 1: Hybrid Edge/IoT + Event-Driven Microservices architecture for the Smart Water Reservoir Management Platform.

5.1 Layer-by-Layer Overview

- **Field / Edge Layer:** Reservoir sensors, weather stations, satellite feeds, and SCADA systems capture raw physical measurements at the source.
- **Ingestion & Streaming Layer:** Edge gateways pre-process data locally; an IoT/MQTT broker and event bus (Kafka-style) transport telemetry to the core platform.
- **Core Microservices Layer:** Independently deployable services handle monitoring, weather analytics, demand forecasting, distribution optimization, drought/flood alerting, orchestration, and reporting.
- **AI/ML Analytics Layer:** Hosts time-series forecasting models and the prescriptive optimization engine consumed by the core services.

- **Data & Storage Layer:** A time-series database, a relational operational database, and a data lake together persist telemetry, transactional data, and historical training data.
- **Application / Presentation Layer:** Admin dashboards, a public transparency portal, a mobile app for field officers, and a report/alert generator present insights to each user group.

6. Components and Their Interactions

6.1 Component Responsibilities

Component	Responsibility
Reservoir/Flow Sensors, Weather Stations, Satellite Feeds, SCADA	Capture raw reservoir level, inflow/outflow, rainfall, and remote-sensing data at the source.
Edge Gateway	Performs local pre-processing/filtering of sensor data to reduce bandwidth and latency before transmission.
IoT Gateway / MQTT Broker	Collects lightweight telemetry from sensors and gateways and publishes it onto the event bus.
API Gateway	Single entry point for external and client requests; handles routing, authentication, and rate limiting.
Event Bus (Kafka-style)	Decouples data producers from consumers; distributes reservoir, weather, and demand events to relevant microservices.
Reservoir Monitoring Service	Tracks live reservoir levels and inflow/outflow against safe operating thresholds.
Rainfall & Weather Analytics Service	Processes rainfall and satellite data to detect patterns feeding the forecasting models.
Demand Forecasting Service	Predicts short/long-term water demand using historical and live consumption data.
Distribution Optimization Service	Uses the optimization engine to recommend allocation and release strategies.
Drought Planning & Alert Service	Evaluates drought/flood risk and triggers alerts through the notification microservice.

Component	Responsibility
Reporting Service	Aggregates data into scheduled water-management reports for authorities.
Orchestration / Workflow Microservice	Coordinates multi-step processes spanning several services (e.g., forecast → optimize → alert → report).
Authentication & Authorization Service	Issues and validates credentials; enforces role-based access across all client-facing endpoints.
AI/ML Analytics Layer	Hosts time-series forecasting models and the prescriptive optimization engine used by the core services.
Data & Storage Layer	Time-series DB for sensor data, relational DB for operational/user data, and a data lake for historical/ML training data.
Application Layer	Admin dashboard, public portal, mobile app, and report/alert generator that present insights to different user groups.

6.2 Interaction and Workflow

- Sensors, SCADA, and satellite feeds continuously publish data via the Edge Gateway and IoT/MQTT broker onto the event bus; client and admin requests instead enter through the API Gateway.
- The event bus fans out reservoir, weather, and demand events to the relevant microservices (monitoring, weather analytics, forecasting) asynchronously, so producers and consumers scale independently.
- The Demand Forecasting and Distribution Optimization services call the AI/ML layer, which reads recent time-series data and historical data-lake records to generate predictions and recommendations.
- Results are persisted to the data layer and passed to the Orchestration service, which sequences the drought-planning/alerting and reporting steps.
- The Drought Planning & Alert Service pushes critical alerts through the Notification microservice to SMS/email/push gateways and to authority dashboards.

- Authorities, the public, and field officers consume outputs via the dashboard, public portal, mobile app, and open-data API, all authenticated through the Authentication & Authorization microservice.

6.3 Representative Sequence: Drought Alert Generation

A concrete walk-through illustrates how the components cooperate end-to-end during a drought-risk scenario:

- 1. Reservoir sensors report falling water levels; the Edge Gateway batches and forwards readings to the IoT broker.
- 2. The event bus delivers the readings to the Reservoir Monitoring Service and the Rainfall & Weather Analytics Service in parallel.
- 3. The Demand Forecasting Service combines the latest readings with historical consumption to update its short-term forecast.
- 4. The Distribution Optimization Service queries the AI/ML layer for a revised release recommendation under constrained supply.
- 5. The Drought Planning & Alert Service evaluates the combined risk score against configured thresholds and raises an alert.
- 6. The Orchestration service sequences notification delivery and triggers the Reporting Service to log the event for audit.
- 7. Authorities receive the alert on the dashboard and mobile app; the public portal reflects an updated supply outlook.

7. Data Architecture

7.1 Data Domains

Data Domain	Examples	Storage
Telemetry / Time-series	Reservoir level, inflow, outflow, rainfall readings	Time-series database (e.g., InfluxDB / TimescaleDB)
Operational / Transactional	Users, roles, alerts, audit logs, reservoir metadata	Relational database (e.g., PostgreSQL)
Historical / ML Training	Multi-year telemetry, satellite imagery, forecasts vs. actuals	Data lake (e.g., object storage + partitioned tables)
Reference / Configuration	Threshold rules, reservoir topology, user roles	Relational database with cached configuration service

7.2 Data Flow Summary

Telemetry flows from field devices through the edge and ingestion layers into the event bus, where it is simultaneously written to the time-series database and consumed by streaming analytics jobs. Forecasting and optimization services read from both the time-series store (recent data) and the data lake (historical context), writing results back to the operational database. Reports and dashboards read exclusively from the operational database and cached forecast outputs, isolating end-user query load from the high-write-throughput ingestion path.

7.3 Data Retention and Quality

- Raw telemetry is retained in the time-series store for a rolling operational window (e.g., 90 days) before being archived to the data lake.
- Data lake records are retained long-term to support multi-year trend analysis and model retraining.
- Automated data-quality checks flag sensor drift, dropped readings, and out-of-range values before they reach forecasting models.
- A data lineage record tracks which raw inputs contributed to each forecast, supporting auditability of automated recommendations.

8. Technology Stack

Layer	Representative Technologies
Edge / IoT	MQTT, LoRaWAN/cellular gateways, edge pre-processing (lightweight containers or firmware)
Event Streaming	Apache Kafka or equivalent managed event-bus service
API Gateway	Kong / NGINX / cloud-native API gateway with OAuth2 support
Microservices Runtime	Containerized services (Docker) orchestrated via Kubernetes
AI/ML Analytics	Python-based forecasting (e.g., Prophet, LSTM/temporal models) and optimization (e.g., linear/mixed-integer programming)
Time-series Storage	TimescaleDB or InfluxDB
Operational Storage	PostgreSQL
Data Lake	Cloud object storage (e.g., S3-compatible) with a partitioned table format
Application Layer	Web dashboard (React), mobile app (cross-platform), report generator
Observability	Centralized logging, metrics, and distributed tracing across all microservices
CI/CD	Automated build, test, and deployment pipelines per microservice

The stack favors managed, horizontally scalable components over bespoke infrastructure wherever possible, reducing operational burden and allowing the platform team to focus engineering effort on the forecasting and optimization logic that differentiates the platform.

9. Discussion of Quality Attributes

Attribute	How the Architecture Addresses It
Scalability	Each microservice and the event bus scale horizontally and independently — e.g., ingestion can scale for peak monsoon telemetry while forecasting scales separately for batch ML jobs.
Security	API Gateway centralizes authentication/authorization; a dedicated Auth microservice enforces role-based access; SCADA integration is isolated behind the edge/ingestion boundary to limit exposure of control systems.
Performance	The event-driven backbone enables near real-time processing of sensor and rainfall data; edge pre-processing reduces payload size and end-to-end latency.
Reliability	Loose coupling via the event bus means a failure in one microservice (e.g., reporting) does not stop data capture or alerting; message queues buffer data during transient outages.
Maintainability	Independently deployable microservices allow forecasting models or the optimization engine to be updated/retrained without redeploying the entire platform.
Availability	Redundant, horizontally scaled critical services (monitoring, alerting) combined with queue-based buffering keep the system responsive during partial failures or network interruptions in remote reservoir sites.
Usability	Role-specific dashboards translate complex forecasts into clear, actionable views for authority staff, field officers, and the public.
Interoperability	The API Gateway and open-data API expose standard, versioned interfaces for meteorological, irrigation-board, and disaster-management integrations.

10. Security Architecture

10.1 Threat Considerations

Because the platform touches SCADA-adjacent infrastructure and government data, its security design treats reservoir control interfaces, data exchange channels, and public-facing endpoints as distinct trust zones.

- **Perimeter security:** The API Gateway terminates TLS, enforces rate limiting, and screens traffic before it reaches internal services.
- **Authentication and authorization:** A dedicated Authentication & Authorization microservice issues short-lived tokens and enforces role-based access control (RBAC) for administrators, field officers, and government agencies.
- **SCADA isolation:** SCADA and control-adjacent integrations sit behind the edge/ingestion boundary in a segregated network zone, never directly exposed to the public-facing application layer.
- **Encryption:** Data is encrypted in transit (TLS) between all services and at rest in the time-series database, operational database, and data lake.
- **Auditability:** All administrative actions and automated recommendations are logged with an immutable audit trail to support regulatory and post-incident review.
- **Secrets management:** Service credentials and API keys are stored in a dedicated secrets manager rather than in application configuration.

10.2 Compliance Considerations

Government and irrigation-board data-sharing agreements typically require access logging, data residency guarantees, and the ability to revoke third-party access quickly. The API Gateway's centralized authentication and the operational database's audit trail together provide the control points needed to satisfy these requirements without embedding compliance logic in every microservice.

11. Deployment Architecture

11.1 Environment Strategy

The platform follows a three-tier environment strategy — development, staging, and production — with automated CI/CD pipelines promoting containerized microservices through each stage after passing automated tests.

11.2 Deployment Topology

- Edge gateways are deployed physically at or near each reservoir site, with local buffering for intermittent connectivity.
- Core microservices, the event bus, and data stores run in a cloud or hybrid-cloud Kubernetes cluster, enabling elastic scaling.
- Critical services (monitoring, alerting) run with multiple replicas across availability zones to avoid single points of failure.
- Batch ML forecasting jobs run on a separately scaled node pool so heavy training workloads do not compete with real-time services for resources.

11.3 Disaster Recovery

The event bus and time-series database are configured with replication across availability zones. Regular backups of the operational database and data lake, combined with infrastructure-as-code definitions for the Kubernetes cluster, allow the platform to be restored in a secondary region within a defined recovery time objective (RTO) and recovery point objective (RPO).

12. Risk Analysis and Mitigation

Risk	Impact	Mitigation
Sensor or connectivity failure at remote reservoirs	Loss of real-time visibility; delayed alerts	Edge buffering, offline-tolerant capture, health-check alerts on sensor gaps
Forecasting model drift as climate patterns shift	Declining forecast accuracy over time	Scheduled retraining pipelines and continuous accuracy monitoring against actuals
Event bus overload during extreme weather spikes	Delayed processing of critical telemetry	Independent horizontal scaling of ingestion and consumer services; topic partitioning
Unauthorized access to SCADA-adjacent systems	Safety and operational risk	Network segregation, RBAC, and dedicated authentication microservice
Data quality issues from legacy or third-party sensors	Inaccurate forecasts and recommendations	Automated data-quality validation before ingestion into forecasting models
Vendor lock-in on managed cloud services	Reduced flexibility, cost risk	Preference for open standards (Kafka-compatible APIs, containerized services)
Single point of failure in orchestration workflow	Cascading delay of alerts and reports	Redundant orchestration instances and idempotent, replayable workflow steps

13. Testing and Validation Strategy

- **Unit testing:** Each microservice maintains automated unit tests covering its core business logic, run on every commit via the CI pipeline.
- **Integration testing:** Contract tests validate event schemas on the event bus and API contracts at the gateway, catching breaking changes before deployment.
- **Model validation:** Forecasting and optimization models are back-tested against historical data and validated against holdout periods before being promoted to production.
- **Load and stress testing:** Ingestion and event-bus components are load-tested against simulated peak-monsoon telemetry volumes to validate scaling assumptions.
- **Failure-injection testing:** Chaos-style tests simulate sensor dropouts, service crashes, and network partitions to validate that monitoring and alerting degrade gracefully rather than failing silently.
- **User acceptance testing:** Reservoir authorities and field officers validate dashboards, alerts, and reports against real operational scenarios before go-live.
- **Security testing:** Periodic penetration testing and dependency vulnerability scanning cover the API Gateway, authentication service, and public-facing endpoints.

14. Justification of Architectural Decisions

14.1 Rationale

The architecture was chosen to match the operational reality of reservoir management: many geographically distributed, intermittently connected sensors producing continuous streams of time-sensitive data that must feed both real-time alerting and longer-cycle ML forecasting. Separating these concerns into an edge layer, an event-streaming backbone, and independent microservices lets each concern be engineered, scaled, and evolved on its own timeline.

14.2 Trade-offs Considered

- **Complexity vs. flexibility:** Microservices and event streaming add operational complexity (service discovery, message-bus management) compared to a monolith, but this is accepted in exchange for independent scalability and fault isolation, both essential given the safety-relevant nature of drought/flood alerts.
- **Eventual consistency vs. strict consistency:** Asynchronous event flow means dashboards may show data that is a few seconds old rather than instantaneous; this is acceptable for reservoir management, where minute-level freshness is sufficient, in exchange for higher throughput and resilience.
- **Operational cost vs. future-proofing:** Running an event bus and multiple services costs more to operate than a single application, but positions the platform to add new data sources (e.g., additional reservoirs, new sensor types) with minimal disruption.

14.3 Support for Future Enhancements and Long-Term Maintainability

- New sensor types or additional reservoirs can be onboarded by extending the edge/ingestion layer alone, without modifying core services.
- New analytical capabilities (e.g., groundwater integration, climate-change scenario modelling) can be added as new microservices subscribing to existing event topics.
- The API Gateway and open-data API allow integration with future government or third-party systems without exposing internal service boundaries.

- Because services are independently deployable, forecasting models can be retrained and redeployed frequently as more historical data accumulates, keeping predictions accurate over time.

15. Future Enhancements and Roadmap

Phase	Enhancement	Value
Near-term	Groundwater-level integration alongside surface reservoirs	More complete picture of total water availability
Near-term	Mobile offline mode with delayed sync for field officers	Reliable data capture at low-connectivity sites
Mid-term	Canal-automation module for remote gate control	Faster execution of optimization recommendations
Mid-term	Climate-change scenario modelling in the optimization engine	Longer-horizon drought and supply planning
Long-term	Cross-reservoir, basin-level optimization	Coordinated allocation across interconnected reservoir systems
Long-term	Citizen-facing conservation recommendations via the public portal	Demand-side reduction to complement supply-side optimization

16. Conclusion

The proposed Hybrid Edge/IoT plus Event-Driven Microservices architecture directly addresses the problem statement's core requirement: replacing manual, historical-observation-based reservoir management with continuous, AI-driven monitoring, forecasting, and decision support. By isolating high-frequency telemetry capture from computation-heavy forecasting and optimization, and by exposing all functionality through a secured API Gateway and role-appropriate application layer, the design achieves the scalability, reliability, security, and maintainability required to support reservoir authorities in efficient water allocation, drought preparedness, and sustainable long-term resource planning.

The architecture is deliberately extensible: new reservoirs, sensor types, analytical models, and integrations can be added incrementally without redesigning the core

platform, positioning it to remain effective as climate variability and demand patterns continue to evolve.

17. Appendix A: Glossary of Terms

Term	Definition
SCADA	Supervisory Control and Data Acquisition — systems used to monitor and control field equipment such as reservoir gates and pumps.
MQTT	A lightweight publish/subscribe messaging protocol commonly used for IoT telemetry.
Event Bus	A messaging backbone (e.g., Kafka) that decouples data producers from consumers via published topics.
Time-series Database	A database optimized for storing and querying timestamped measurements such as sensor readings.
Data Lake	Large-scale storage for raw and historical data used for analytics and ML model training.
RBAC	Role-Based Access Control — restricting system access according to a user's assigned role.
RTO / RPO	Recovery Time Objective / Recovery Point Objective — targets for how quickly and how completely a system recovers after an outage.

18. Appendix B: Representative Use Cases

18.1 Use Case — Drought Scenario Planning

A reservoir authority requests a 90-day drought scenario simulation ahead of a forecast dry season. The platform's Distribution Optimization Service, informed by the AI/ML layer's demand and availability forecasts, generates recommended release reductions across drinking, irrigation, and industrial allocations, allowing the authority to compare trade-offs before committing to a plan.

18.2 Use Case — Sensor Fault Detection

A reservoir-level sensor begins reporting anomalous, flat-lined readings due to a hardware fault. Automated data-quality checks in the ingestion pipeline flag the anomaly, the Reservoir Monitoring Service suppresses the faulty readings from

downstream forecasts, and an alert is routed to field officers to schedule a maintenance visit.

18.3 Use Case — Public Transparency Reporting

A citizen visits the public portal to check current reservoir status ahead of the irrigation season. The portal calls the API Gateway, which routes the request to the Reporting Service; cached, non-sensitive aggregate data is returned without exposing raw SCADA-adjacent telemetry, balancing transparency with operational security.

References

1. Problem Statement 34 — AI-Powered Smart Water Reservoir Management and Demand Forecasting Platform, CO2 Assessment Tool 2, CSA10 Software Engineering course brief.
2. Course materials on scalable software architecture styles: layered, microservices, and event-driven architectures, CSA10 — Software Engineering.
3. General industry practice references for IoT/edge computing, time-series data management, and stream-processing architectures used to inform the technology stack in Section 8.